1. Experiment No. :Experiment No.: 05
2. Student Name : ADESH DEEPAK PATANKAR
3.
4. Class : TYBDCE

EXPERIMENT NO. 05

Title: Design and implement a MapReduce program to identify the maximum temperature recorded for a specified year from a large weather dataset.

Outcome: Students will be able to implement a program to find the maximum temperature of a particular year in an input file.

Theory:

Analyzing the Data with Hadoop Using Map Reduce

To take advantage of the parallel processing that Hadoop provides, we need to express

our query as a MapReduce job because MapReduce framwork will manage the parallel processing by it self.

MapReduce divides the processing into 2 phases -

1. the map phase
2. the reduce phase

Each phase has input in the form of key value pair, and both phases produces output as key value pair.

The output that has been generated by the map phase is given to a reduce phase as an input.

It is the programmers responsibility to specifies two functions:

1. the map function
2. the reduce function

Lets take an example where input to the map phase is from the below link which has the NCDC data-

https://raw.githubusercontent.com/lmsamarawickrama/Hadoop-MapReduce/master/NCDC%20weather%20files/1901

Using the above data we need to calculate maximum temperature per year.

While writing a mapreudce code, We choose a text input format (which is also the default format for hadoop) that

gives us each line in the dataset as a text value. The key is the offset of the beginning of

the line from the beginning of the file, but as we have no need for this, we ignore it.

Since we are only concerned about the year and the temperature our map function is simple. We pull out the year and the air temperature. In this case, the map function is just a data

preparation phase, setting up the data in such a way that the reduce function can do

its work on it: finding the maximum temperature for each year. The map function is

also a good place to drop bad records for example removing the bad or invalid

records.

Consider map gets below lines as an input -

0067011990999991950051507004...9999999N9+00001+99999999999...

0043011990999991950051512004...9999999N9+00221+99999999999...

0043011990999991950051518004...9999999N9-00111+99999999999...

0043012650999991949032412004...0500001N9+01111+99999999999...

0043012650999991949032418004...0500001N9+00781+99999999999...

These lines are presented to the map function as the key-value pairs:

(0, 006701199099999 1950051507004...9999999N9+00001+99999999999...)

(106, 004301199099999 1950051512004...9999999N9+00221+99999999999...)

(212, 004301199099999 1950051518004...9999999N9-00111+99999999999...)

(318, 004301265099999 1949032412004...0500001N9+01111+99999999999...)

(412, 004301265099999 1949032418004...0500001N9+00781+99999999999...)

The keys (0,106,212 etc in our example) are the line offsets within the file, which we ignore in our map function. The

map function will simply extract the year and the temperature (indicated in bold text),

and emits them as its output.

Below is the sample output

(1950, 0)

(1950, 22)

(1950, −11)

(1949, 111)

(1949, 78)

Now Mapreduce framework takes the output generated by the map function and processes it before sending it to the reducer. Framework will sort the records based on the keys generated by the mapper and after sorting it will group all the values associated with single key as one record for reducer.

So our reduce function sees the following input:

(1949, [111, 78])

(1950, [0, 22, −11])

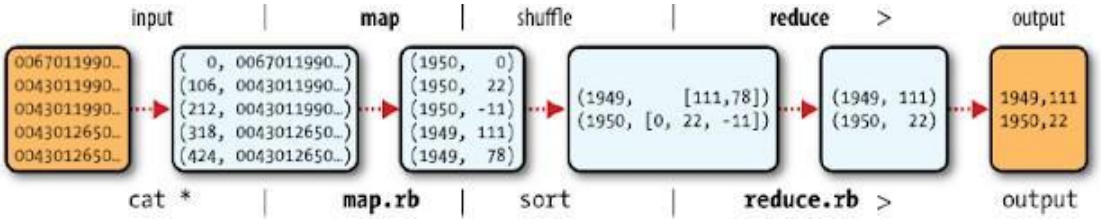
Here, We have got only 2 records instead of 4 (output of map function has 4 output records) and it is in sorted order.

Each year appears with a list of all its air temperature readings. All the reduce function has to do now is iterate through the list and pick up the maximum value by iterating the list. Below is the output of the reducer function.

(1949, 111)

(1950, 22)

The graphical data flow is illustrated in below figure -



dataset sample.txt

0067011990999991950051507004+68750+023550FM-12+038299999V0203301N00671220001CN9999999N9+00001+99999999999

0043011990999991950051512004+68750+023550FM-12+038299999V0203201N00671220001CN9999999N9+00221+99999999999

0043011990999991950051518004+68750+023550FM-12+038299999V0203201N00261220001CN9999999N9-00111+99999999999

0043012650999991949032412004+62300+010750FM-12+048599999V0202701N00461220001CN0500001N9+01111+99999999999

0043012650999991949032418004+62300+010750FM-12+048599999V0202701N00461220001CN0500001N9+00781+99999999999

MaxDriver.java

```
import org.apache.hadoop.fs.Path;
import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;

public class MaxDriver {
        public static void main(String args[])throws Exception{
                if(args.length!=2){
                        System.out.println("input valid
                        arg"); System.exit(-1);
                }
                JobConf conf=new JobConf(MaxDriver.class);
                conf.setMapperClass(MaxMapper.class);
                conf.setReducerClass(MaxReducer.class);
                conf.setOutputKeyClass(Text.class);
                conf.setOutputValueClass(IntWritable.class);
                FileInputFormat.addInputPath(conf, new Path(args[0]));
                FileOutputFormat.setOutputPath(conf, new Path(args[1]));
                JobClient.runJob(conf);

        }

}
```

MaxMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

public class MaxMapper extends MapReduceBase implements Mapper<LongWritable,Text,Text,IntWritable>{

	public void map(LongWritable key, Text value,
			OutputCollector<Text, IntWritable> values, Reporter r)
			throws IOException {
		String s=value.toString();
		String s1=s.substring(15, 19);
		int temperature;

		if(s.charAt(87)=='+'){
			temperature=Integer.parseInt(s.substring(88,92));
		} else{
			temperature=Integer.parseInt(s.substring(87,92));
		}
		values.collect(new Text(s1),new IntWritable(temperature));
	}
}
```

MaxReducer.java

```
import
java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapred.MapReduceBase;
import  org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class MaxReducer extends MapReduceBase
implements Reducer<Text,IntWritable,Text,IntWritable>{

                public void reduce(Text key, Iterator<IntWritable> value1,
                        OutputCollector<Text, IntWritable> values, Reporter r)
                        throws IOException {

                        int maxvalue=Integer.MIN_VALUE;
                        while(value1.hasNext()){
                                maxvalue=Math.max(maxvalue, value1.next().get());
                        }
                        values.collect(key, new IntWritable(maxvalue));
        }

}
```

Steps needed to run the project

1. `Open Eclipse`

File-New Java Project

Right click: New Java file:

Write Mapreduce code then save it.

2. `Import jar files`

Right click on project->Properties->java build path->add external jars

3. `Export jar file`

Right click on project->Export

4. `Adding input file to HDFS`

$hadoop fs -put <filename>

5. `Running`

$hadoop jar <program name> <input> <output>

Ex: $hadoop jar wordcount.jar wordcount /output

Commands:-

start-dfs.sh

start-yarn.sh

jps

hdfs dfs -ls /

hdfs dfs -mkdir /Ex5Input

hdfs dfs -copyFromLocal /home/hadoop/Downloads/Maxtemp_Year_Hadoop_MapReduce-master/dataset.txt /Ex5Input

hadoop jar /home/hadoop/Ex5temp.jar com.tem.MaxDriver /Ex5Input /ex5result3

hadoop fs -cat /ex4/part-r-00000

Handwritten Questions (for Assignment / Viva)

1. What is the role of the Mapper and the Reducer in the maximum temperature program?
2. Explain how the shuffle and sort phase works in MapReduce with respect to temperature data.
3. Why do we need HDFS to store large temperature datasets instead of a local file system?
4. Modify the program logic to find the minimum temperature of each year instead of maximum.

MIT SCHOOL OF COMPUTING
Rajbaug, Loni-Kalbhor, Pune
MIT-ADT UNIVERSITY
PUNE, INDIA
A Leap Towards World Class Education

Dhruv. A. Sakhare
BDTL
Roll no :- 13
BDCE

Experiment - 5

Q1 What is the role of the Mapper and Reducer in the maximum temperature program?

⇒ The mapper reads the temperature data and generates key value pair in form of (Year, Temperature). The reducer receives all temperatures for the same year and finds the maximum temperature.

Q2 Explain how the shuffle and sort phase works in MapReduce with respect to temperature data?

⇒ The shuffle and sort phase groups all temperature values having the same year and sorts them by key. For example 1950 → [22, 25, 20] These grouped values are then sent to the Reducer.

Q3 Why do we need HDFS to store large dataset temperature instead of local file system.

⇒ HDFS is used because temperature dataset can be very large. It stores data across multiple machines provides fault tolerance.

through replication and allows MapReduce to process data in parallel.

Q. Modify the program logic to find the minimum temperature of each year instead of maximum?

⇒ The mapper remains the same In reducer replace the maximum-temperature logic with minimum temperature logic

```
double minTemp = Double.MAX_VALUE;

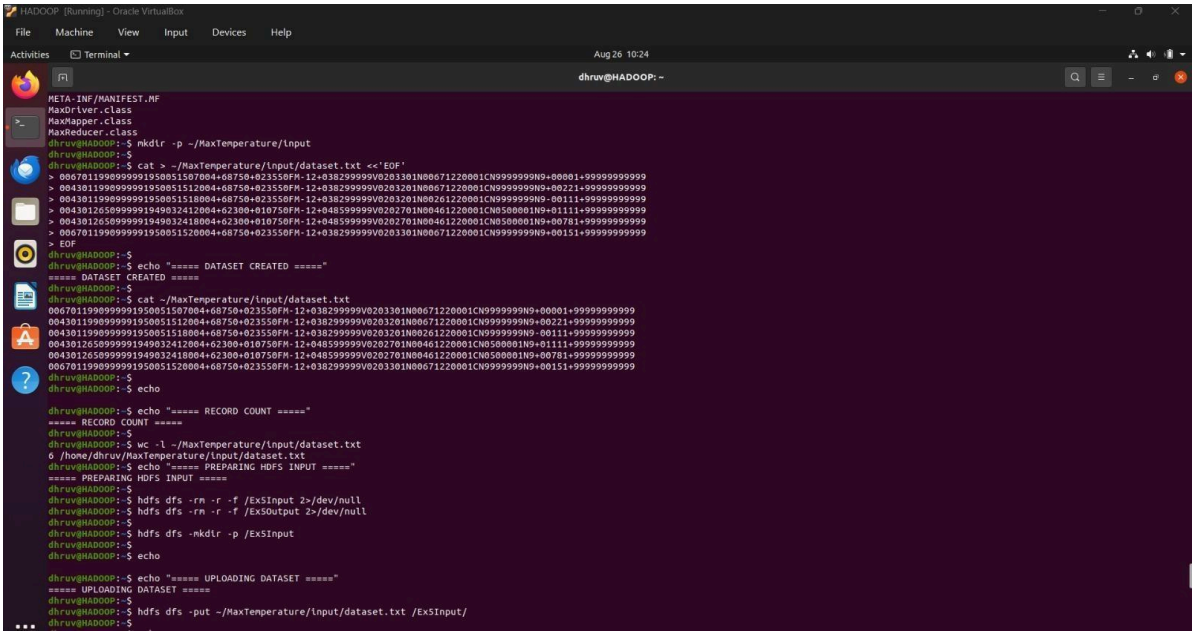
for (DoubleTemp : values) {
    minTemp = Math.min(MinTemp, temp);
}

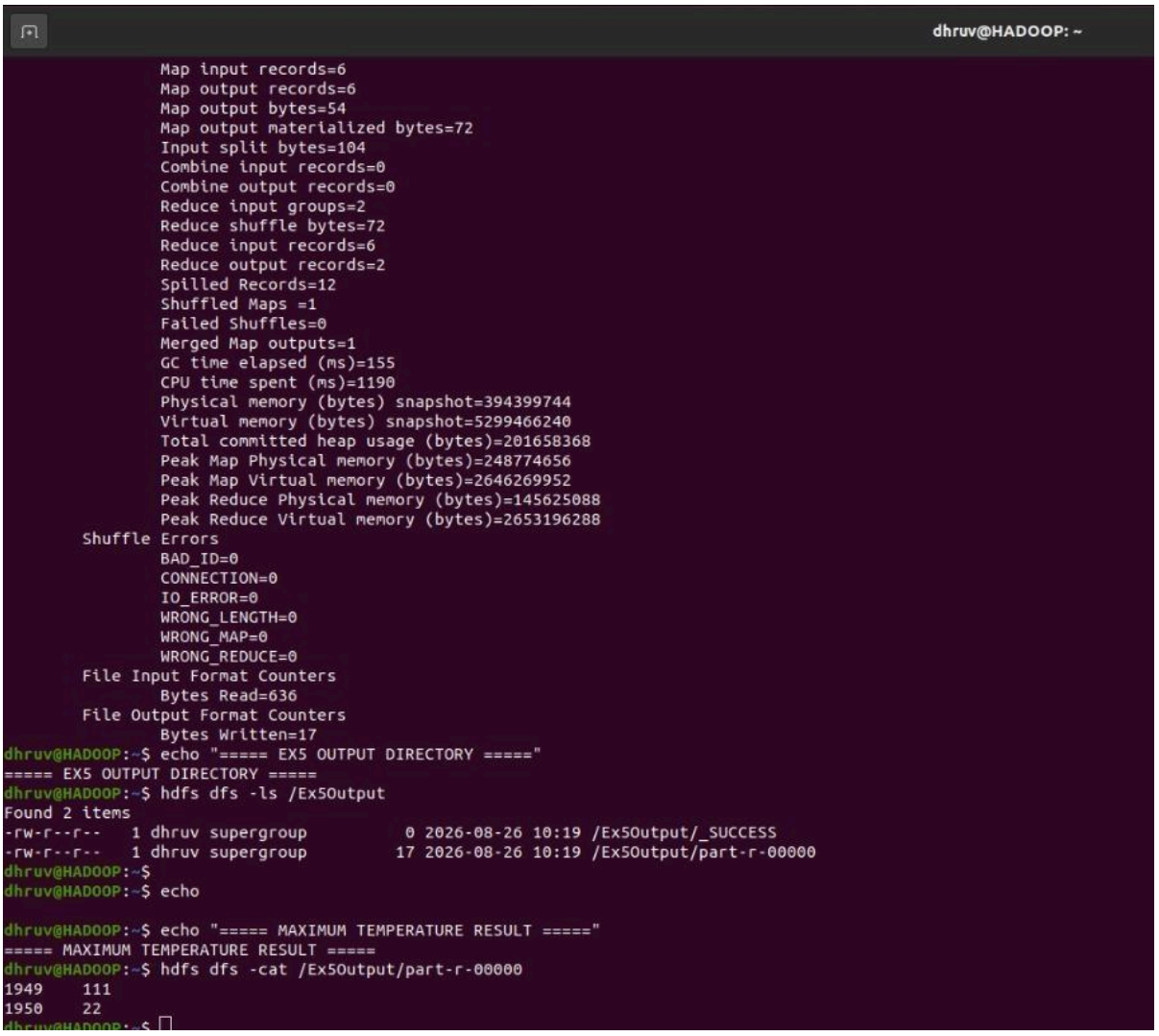
context.write(key, new Text(String.valueOf(minTemp)));
```

This will produce minimum temperature of each year.

Acceptanec Criteria:

5. Program must correctly read the **input temperature dataset** from HDFS.
6. Mapper should emit **<year, temperature>** pairs for each record.
7. Reducer must output the **maximum temperature per year** correctly.
8. Final output should be stored in a specified **HDFS output directory** and verified with hdfs dfs -cat.





Conclusion: Thus, I have implemented a program to find the maximum temperature of a particular year in an input file.